



UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – PROJETO DE SISTEMAS DIGITAIS
PROF. PhD. DOUGLAS ROSSI DE MELO

GUSTAVO HENRIQUE STAHL MÜLLER

PROJETO DE CIRCUITOS COMBINACIONAIS USANDO VHDL

ITAJAÍ
2022

SUMÁRIO

1. INTRODUÇÃO.....	3
2. DESENVOLVIMENTO.....	4
2.1. MULTIPLEXADOR 2-PARA-1	4
2.2. DEMULTIPLEXADOR 1-PARA-2.....	7
2.3. CODIFICADOR 4-PARA-2	11
2.4. DECODIFICADOR 2-PARA-4.....	16

1. INTRODUÇÃO

Esse trabalho tem como objetivo consolidar os conhecimentos de projeto de circuitos combinacionais, envolvendo a criação dos quatro seguintes circuitos usando as ferramentas VHDL e EDA Playground:

1. Um multiplexador 2-para-1.
2. Um demultiplexador 1-para-2.
3. Um codificador binário 2-para-4.
4. Um decodificador binário 4-para-2;

2. DESENVOLVIMENTO

2.1. MULTIPLEXADOR 2-PARA-1

Um multiplexador é um circuito combinacional com n entradas e com somente uma saída, que possui o objetivo de redirecionar uma entrada específica até a saída com base em um seletor. O tamanho do seletor depende da quantidade de entradas.

Figura 1 – Tabela verdade do multiplexador 2-para-1.

A	B	SEL	S
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Fonte: O autor.

O multiplexador 2-para-1 descrito na especificação do trabalho foi usado. O valor 0 do seletor significa a escolha da entrada “A”, enquanto o valor 1 significa a escolha da entrada “B”.

Figura 2 – Entidade do multiplexador 2-para-1.

```
library ieee;
use ieee.std_logic_1164.all;

entity mux2_1bit is
port ( i_SEL : in std_logic; -- selector
      i_A : in std_logic; -- data input
      i_B : in std_logic; -- data input
      o_S : out std_logic); -- data output
end mux2_1bit;
```

Fonte: O autor.

Figura 3 – Arquitetura do multiplexador 2-para-1.

```
architecture arch_1 of mux2_1bit is
begin
  process(i_SEL, i_A, i_B)
  begin
    if (i_SEL='0') then
      o_S <= i_A;
    else
```

```

        o_S <= i_B;
    end if;
end process;
end arch_1;

```

Fonte: O autor.

Figura 4 – Testbench do multiplexador 2-para-1.

```

library ieee;
use ieee.std_logic_1164.all;

entity tb_mux2_1bit is
-- empty
end tb_mux2_1bit;

architecture arch_1 of tb_mux2_1bit is

-- DUT component
component mux2_1bit is
port ( i_SEL : in std_logic; -- selector
i_A : in std_logic; -- data input
i_B : in std_logic; -- data input
o_S : out std_logic); -- data output
end component;

signal w_SEL, w_A, w_B, w_S : std_logic;

begin
-- Connect DUT
u_DUT: mux2_1bit port map(i_SEL => w_SEL, i_A => w_A, i_B => w_B,
    o_S => w_S);

process
begin
-- Select input channel A
w_SEL <= '0';
w_A <= '0';
w_B <= '0';
wait for 1 ns;
assert(w_S='0') report "Fail @ 000" severity error;
w_SEL <= '0';
w_A <= '0';
w_B <= '1';
wait for 1 ns;

```

```

    assert(w_S='0') report "Fail @ 001" severity error;
    w_SEL <= '0';
    w_A <= '1';
    w_B <= '0';
    wait for 1 ns;
    assert(w_S='1') report "Fail @ 010" severity error;
    w_SEL <= '0';
    w_A <= '1';
    w_B <= '1';
    wait for 1 ns;
    assert(w_S='1') report "Fail @ 011" severity error;
    -- Select input channel B
    w_SEL <= '1';
    w_A <= '0';
    w_B <= '0';
    wait for 1 ns;
    assert(w_S='0') report "Fail @ 100" severity error;
    w_SEL <= '1';
    w_A <= '0';
    w_B <= '1';
    wait for 1 ns;
    assert(w_S='1') report "Fail @ 101" severity error;
    w_SEL <= '1';
    w_A <= '1';
    w_B <= '0';
    wait for 1 ns;
    assert(w_S='0') report "Fail @ 110" severity error;
    w_SEL <= '1';
    w_A <= '1';
    w_B <= '1';
    wait for 1 ns;
    assert(w_S='1') report "Fail @ 111" severity error;
    -- Clear inputs
    w_SEL <= '0';
    w_A <= '0';
    w_B <= '0';
    assert false report "Test done." severity note;
    wait;
end process;
end arch_1;

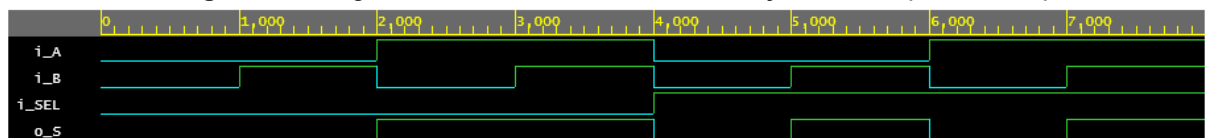
```

Fonte: O autor.

Através do diagrama de forma de onda, é possível ver que o circuito se comporta corretamente:

- No instante 2, a saída é verdadeira pois A é 1 e SEL é 0;
- No instante 3, a saída é verdadeira pois A é 1 e SEL é 0;
- No instante 5, a saída é verdadeira pois B é 1 e SEL é 1;
- No instante 7, a saída é verdadeira pois B é 1 e SEL é 1;
- Em todos os outros instantes, a saída é falsa.

Figura 5 – Diagrama de forma de onda da simulação do multiplexador 2-para-1.



Fonte: O autor.

Todos os testes do testbench executam corretamente sem mensagens de erro:

Figura 6 – Imagem do console após a execução do testbench do multiplexador 2-para-1.

```
# Aldec, Inc. Riviera-PRO version 2022.04.117.8517 built for Linux64 on May 04, 2022.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2022 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Elaboration time 0.0 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... skipped, nothing to simulate in SLP mode : 0.0 [s]
# SLP: Finished : 0.0 [s]
# ELAB2: You do not have a license to run VHDL performance optimized simulation. Contact Aldec for ordering information - sales@aldec.com.
# ELAB2: Elaboration final pass complete - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 5401 kB (elbread=427 elab2=4831 kernel=142 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsysstf.so'
# EXECUTION:: NOTE : Test done.
# EXECUTION:: Time: 8 ns, Iteration: 0, Instance: /tb_mux2_1bit, Process: line__28.
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2022-09-17 21:20:11 EDT] Opening EPWave...
Done
```

Fonte: O autor.

2.2. DEMULTIPLEXADOR 1-PARA-2

Um demultiplexador é o circuito inverso do multiplexador, ou seja, mapeia somente uma entrada à n saídas através de um seletor. O seletor indica em qual saída será redirecionada o valor da entrada.

Figura 6 – Tabela verdade do demultiplexador 2-para-1.

A	SEL	S(0)	S(1)
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

Fonte: O autor.

O valor de seletor igual à 0 significa a escrita de A na saída S(0) e o valor igual à 1 significa a escrita de A na saída S(1). As equações booleanas que correspondem aos valores de S(0) e S(1) são:

Figura 6 – Equações booleanas do demultiplexador 1-para-2.

$$S(0) = A \wedge SEL'; \quad S(1) = A \wedge SEL;$$

Fonte: O autor.

Após definir o comportamento esperado do circuito através da tabela verdade e a sua implementação através das equações booleanas, é possível codificar a entidade, arquitetura e testbench:

Figura 7 – Entidade do demultiplexador 1-para-2.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity demux1_2 is
port
(
    i_A: in std_logic;
    i_SEL: in std_logic;
    o_S: out std_logic_vector (1 downto 0)
);
end demux1_2;
```

Fonte: O autor.

Figura 8 – Arquitetura do demultiplexador 1-para-2.

```
architecture arch_demux1_2 of demux1_2 is
begin
    o_S(0) <= i_A and not i_SEL;
    o_S(1) <= i_A and i_SEL;
end arch_demux1_2;
```

Fonte: O autor.

Figura 9 – Testbench do demultiplexador 1-para-2.

```
library IEEE;
use IEEE.std_logic_1164.all;
```



```

entity demux1_2_tb is
end demux1_2_tb;

architecture arch_demux1_2_tb of demux1_2_tb is
  component demux1_2 is
    port
      (
        i_A: in std_logic;
        i_SEL: in std_logic;
        o_S: out std_logic_vector (1 downto 0)
      );
  end component;

  signal w_A, w_SEL: std_logic;
  signal w_S: std_logic_vector (1 downto 0);

begin
  demux_instance: demux1_2 port map(w_A, w_SEL, w_S);

  process
  begin
    w_A <= '0';
    w_SEL <= '0';
    wait for 1ns;
    assert (w_S(1) = '0') report "Fail 0/0 == 0 -> output(1) != 0" severity error;
    assert (w_S(0) = '0') report "Fail 0/0 == 0 -> output(0) != 0" severity error;

    w_A <= '0';
    w_SEL <= '1';
    wait for 1ns;
    assert (w_S(1) = '0') report "Fail 0/1 == 0 -> output(1) != 0" severity error;
    assert (w_S(0) = '0') report "Fail 0/1 == 0 -> output(0) != 0" severity error;

    w_A <= '1';
    w_SEL <= '0';
    wait for 1ns;
    assert (w_S(1) = '0') report "Fail 1/0 == 0 -> output(1) != 0" severity error;
    assert (w_S(0) = '1') report "Fail 1/0 == 1 -> output(0) != 1" severity error;

    w_A <= '1';
    w_SEL <= '1';
    wait for 1ns;
    assert (w_S(1) = '1') report "Fail 1/1 == 1 -> output(1) != 1" severity error;
    assert (w_S(0) = '0') report "Fail 1/1 == 0 -> output(0) != 0" severity error;

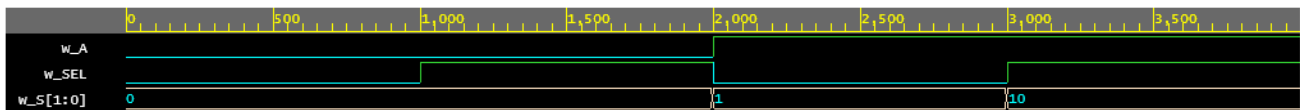
    wait;
  end process;
end arch_demux1_2_tb;

```

Fonte: O autor.

Através do diagrama de forma de onda, é possível ver que o circuito se comporta corretamente:

- A saída é 00 quando A é 0;
- A saída é 01 quando A é 1 e SEL é 0;
- A saída é 10 quando A é 1 e SEL é 1;

Figura 10 – Diagrama de forma de onda da simulação do demultiplexador 1-para-2.

Fonte: O autor.

Todos os testes do testbench executam corretamente sem mensagens de erro:

Figura 11 – Imagem do console após a execução do testbench do demultiplexador 1-para-2.

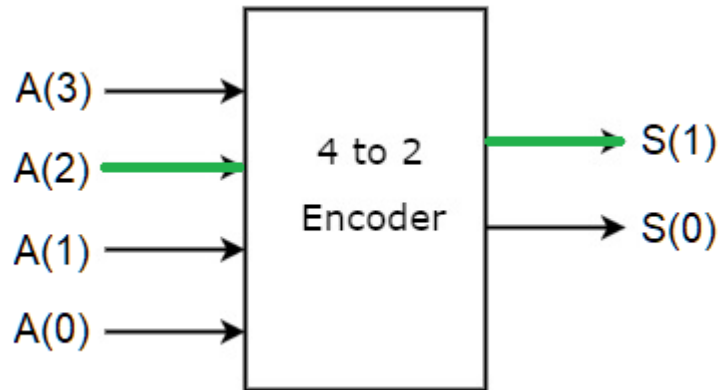
```
# Aldec, Inc. Riviera-PRO version 2022.04.117.8517 built for Linux64 on May 04, 2022.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2022 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Elaboration time 0.0 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... skipped, nothing to simulate in SLP mode : 0.0 [s]
# SLP: Finished : 0.0 [s]
# ELAB2: You do not have a license to run VHDL performance optimized simulation. Contact Aldec for ordering information - sales@aldec.com.
# ELAB2: Elaboration final pass complete - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 6424 kB (elbread=427 elab2=5854 kernel=142 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsysstf.so'
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2022-09-17 21:18:03 EDT] Opening EPwave...
Done
```

Fonte: O autor.

2.3. CODIFICADOR 4-PARA-2

Um codificador é um circuito combinacional que possui n entradas e $\log_2 n$ saídas, com o objetivo de transformar o índice da entrada em um número binário. Por exemplo, no seguinte codificador, a entrada “A(2)” geraria as saídas “10”, representando o número 2.

Figura 12 – Exemplo de uso de um codificador 4-para-2.



Fonte: O autor.

No circuito desenvolvido, apenas as combinações de entradas que possuem somente um valor verdadeiro foram consideradas para a criação das equações booleanas. Logo, qualquer combinação com mais de uma entrada verdadeira resulta em comportamento indefinido.

Figura 13 – Tabela verdade do codificador 4-para-2

A(3)	A(2)	A(1)	A(0)	S(1)	S(0)
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	1
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	1	0
0	1	1	0	1	1
0	1	1	1	1	1
1	0	0	0	1	1
1	0	0	1	1	1
1	0	1	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	1	1

Fonte: O autor.

Índices menores correspondem aos bits menos significativos. $S(0)$ se torna verdadeiro quando $A(1)$ ou $A(3)$ estão selecionados e $S(1)$ se torna verdadeiro quando $A(2)$ ou $A(3)$ estão selecionados. Logo:

Figura 14 – Equações booleanas do codificador 4-para-2.

$$S(0) = A(1) \vee A(3); \quad S(1) = A(2) \vee A(3);$$

Fonte: O autor.

Após definir o comportamento esperado do circuito através da tabela verdade e a sua implementação através das equações booleanas, é possível codificar a entidade, arquitetura e testbench:

Figura 15 – Entidade do codificador 4-para-2.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity codifier4_2 is
port
(
    i_A: in std_logic_vector (3 downto 0);
    o_S: out std_logic_vector (1 downto 0)
);
end codifier4_2;
```

Fonte: O autor.

Figura 16 – Arquitetura do codificador 4-para-2.

```
architecture arch_codifier4_2 of codifier4_2 is
begin
    o_S(0) <= i_A(1) or i_A(3);
    o_S(1) <= i_A(2) or i_A(3);
end arch_codifier4_2;
```

Fonte: O autor.

Figura 17 – Testbench do codificador 4-para-2.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity codifier4_2_tb is
end codifier4_2_tb;

architecture arch_codifier4_2_tb of codifier4_2_tb is
    component codifier4_2 is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        o_S: out std_logic_vector (1 downto 0)
    );
    end component;
end component;
```

```

signal w_A: std_logic_vector (3 downto 0);
signal w_S: std_logic_vector (1 downto 0);

begin
  codifier_instance: codifier4_2 port map(w_A, w_S);

  process
  begin
    w_A(3) <= '0';
    w_A(2) <= '0';
    w_A(1) <= '0';
    w_A(0) <= '0';
    wait for 1ns;
    assert (w_S(1) = '0') report "Fail 0/0/0/0 -> output(1) != 0" severity error;
    assert (w_S(0) = '0') report "Fail 0/0/0/0 -> output(0) != 0" severity error;

    w_A(3) <= '0';
    w_A(2) <= '0';
    w_A(1) <= '0';
    w_A(0) <= '1';
    wait for 1ns;
    assert (w_S(1) = '0') report "Fail 0/0/0/1 -> output(1) != 0" severity error;
    assert (w_S(0) = '0') report "Fail 0/0/0/1 -> output(0) != 0" severity error;

    w_A(3) <= '0';
    w_A(2) <= '0';
    w_A(1) <= '1';
    w_A(0) <= '0';
    wait for 1ns;
    assert (w_S(1) = '0') report "Fail 0/0/1/0 -> output(1) != 0" severity error;
    assert (w_S(0) = '1') report "Fail 0/0/1/0 -> output(0) != 1" severity error;

    w_A(3) <= '0';
    w_A(2) <= '0';
    w_A(1) <= '1';
    w_A(0) <= '1';
    wait for 1ns;
    assert (w_S(1) = '0') report "Fail 0/0/1/1 -> output(1) != 0" severity error;
    assert (w_S(0) = '1') report "Fail 0/0/1/1 -> output(0) != 1" severity error;

    w_A(3) <= '0';
    w_A(2) <= '1';
    w_A(1) <= '0';
    w_A(0) <= '0';
    wait for 1ns;
    assert (w_S(1) = '1') report "Fail 0/1/0/0 -> output(1) != 1" severity error;
    assert (w_S(0) = '0') report "Fail 0/1/0/0 -> output(0) != 0" severity error;

    w_A(3) <= '0';
    w_A(2) <= '1';
    w_A(1) <= '0';
    w_A(0) <= '1';
    wait for 1ns;
    assert (w_S(1) = '1') report "Fail 0/1/0/1 -> output(1) != 1" severity error;
    assert (w_S(0) = '0') report "Fail 0/1/0/1 -> output(0) != 0" severity error;

    w_A(3) <= '0';
    w_A(2) <= '1';
    w_A(1) <= '1';
    w_A(0) <= '0';
    wait for 1ns;
    assert (w_S(1) = '1') report "Fail 0/1/1/0 -> output(1) != 1" severity error;
    assert (w_S(0) = '1') report "Fail 0/1/1/0 -> output(0) != 1" severity error;
  end process;

```

```

w_A(3) <= '0';
w_A(2) <= '1';
w_A(1) <= '1';
w_A(0) <= '1';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 0/1/1/1 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 0/1/1/1 -> output(0) != 1" severity error;

w_A(3) <= '1';
w_A(2) <= '0';
w_A(1) <= '0';
w_A(0) <= '0';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 1/0/0/0 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 1/0/0/0 -> output(0) != 1" severity error;

w_A(3) <= '1';
w_A(2) <= '0';
w_A(1) <= '0';
w_A(0) <= '1';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 1/0/0/1 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 1/0/0/1 -> output(0) != 1" severity error;

w_A(3) <= '1';
w_A(2) <= '0';
w_A(1) <= '1';
w_A(0) <= '0';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 1/0/1/0 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 1/0/1/0 -> output(0) != 1" severity error;

w_A(3) <= '1';
w_A(2) <= '0';
w_A(1) <= '1';
w_A(0) <= '1';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 1/0/1/1 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 1/0/1/1 -> output(0) != 1" severity error;

w_A(3) <= '1';
w_A(2) <= '1';
w_A(1) <= '0';
w_A(0) <= '0';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 1/1/0/0 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 1/1/0/0 -> output(0) != 1" severity error;

w_A(3) <= '1';
w_A(2) <= '1';
w_A(1) <= '0';
w_A(0) <= '1';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 1/1/0/1 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 1/1/0/1 -> output(0) != 1" severity error;

w_A(3) <= '1';
w_A(2) <= '1';
w_A(1) <= '1';
w_A(0) <= '0';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 1/1/1/0 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 1/1/1/0 -> output(0) != 1" severity error;

```

```

w_A(3) <= '1';
w_A(2) <= '1';
w_A(1) <= '1';
w_A(0) <= '1';
wait for 1ns;
assert (w_S(1) = '1') report "Fail 1/1/1/1 -> output(1) != 1" severity error;
assert (w_S(0) = '1') report "Fail 1/1/1/1 -> output(0) != 1" severity error;

wait;
end process;
end arch_codifier4_2_tb;

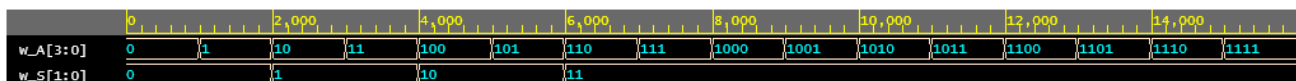
```

Fonte: O autor.

Através do diagrama de forma de onda, é possível ver que o circuito se comporta corretamente:

- A entrada 0001 gera a saída 0 (00);
- A entrada 0010 gera a saída 1 (01);
- A entrada 0100 gera a saída 2 (10);
- A entrada 1000 gera a saída 3 (11);
- Qualquer outra entrada gera um valor indefinido.

Figura 18 – Diagrama de forma de onda da simulação do codificador 4-para-2.



Fonte: O autor.

Todos os testes do testbench executam corretamente sem mensagens de erro:

Figura 19 – Imagem do console após a execução do testbench do codificador 4-para-2.

```

# Aldec, Inc. Riviera-PRO version 2022.04.117.8517 built for Linux64 on May 04, 2022.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2022 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Elaboration time 0.0 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... skipped, nothing to simulate in SLP mode : 0.0 [s]
# SLP: Finished : 0.0 [s]
# ELAB2: You do not have a license to run VHDL performance optimized simulation. Contact Aldec for ordering information - sales@aldec.com.
# ELAB2: Elaboration final pass complete - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 6430 kB (elbread=427 elab2=5860 kernel=142 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsysstf.so'
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2022-09-17 21:16:46 EDT] Opening EPWave...
Done

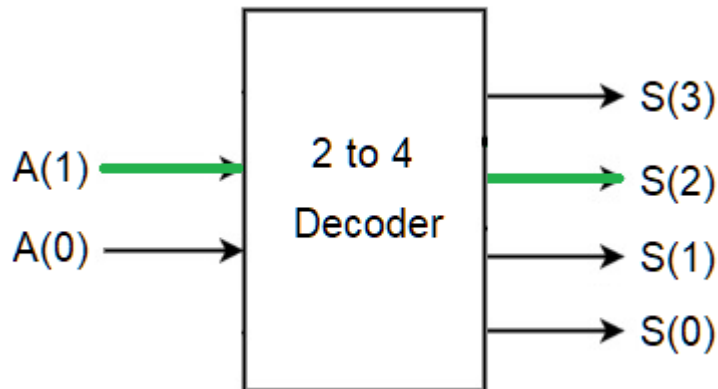
```

Fonte: O autor.

DECODIFICADOR 2-PARA-4

Um decodificador é o circuito inverso do codificador, ou seja, mapeia n entradas à 2^n saídas, com o objetivo de receber um número binário e ativar a saída de índice correspondente. Por exemplo, a entrada $A(1)$, que representa o número 2 (10) ativa como saída $S(2)$.

Figura 20 – Exemplo de uso de um decodificador 2-para-4.



Fonte: O autor.

Figura 21 – Tabela verdade do decodificador 2-para-4.

A(1)	A(0)	S(3)	S(2)	S(1)	S(0)
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Fonte: O autor.

Índices menores correspondem aos bits menos significativos. As equações booleanas das saídas do circuito são:

Figura 22 – Equações booleanas do codificador 4-para-2.

$$\begin{aligned}
 S(0) &= A(0)' \vee A(1)'; & S(1) &= A(0) \vee A(1)'; \\
 S(2) &= A(0)' \vee A(1); & S(3) &= A(0) \vee A(1);
 \end{aligned}$$

Fonte: O autor.

Após definir o comportamento esperado do circuito através da tabela verdade e a sua implementação através das equações booleanas, é possível codificar a entidade, arquitetura e testbench:

Figura 23 – Entidade do decodificador 2-para-4.

```

library IEEE;
use IEEE.std_logic_1164.all;

```



```

entity decodifier2_4 is
port
(
    i_A: in std_logic_vector (1 downto 0);
    o_S: out std_logic_vector (3 downto 0)
);
end decodifier2_4;

```

Fonte: O autor.

Figura 24 – Arquitetura do decodificador 2-para-4.

```

architecture arch_decodifier2_4 of decodifier2_4 is
begin
    o_S(3) <= i_A(1) and i_A(0);
    o_S(2) <= i_A(1) and not i_A(0);
    o_S(1) <= not i_A(1) and i_A(0);
    o_S(0) <= not i_A(1) and not i_A(0);
end arch_decodifier2_4;

```

Fonte: O autor.

Figura 25 – Testbench do decodificador 2-para-4.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity decodifier2_4_tb is
end decodifier2_4_tb;

architecture arch_decodifier2_4_tb of decodifier2_4_tb is
    component decodifier2_4 is
        port
        (
            i_A: in std_logic_vector (1 downto 0);
            o_S: out std_logic_vector (3 downto 0)
        );
    end component;

    signal w_A: std_logic_vector (1 downto 0);
    signal w_S: std_logic_vector (3 downto 0);

begin
    decodifier_instance: decodifier2_4 port map(w_A, w_S);

    process
    begin
        w_A(1) <= '0';
        w_A(0) <= '0';
        wait for 1ns;
        assert (w_S(3) = '0') report "Fail 0/0 -> output(3) == 1" severity error;
        assert (w_S(2) = '0') report "Fail 0/0 -> output(2) == 1" severity error;
        assert (w_S(1) = '0') report "Fail 0/0 -> output(1) == 1" severity error;
        assert (w_S(0) = '1') report "Fail 0/0 -> output(0) != 1" severity error;

        w_A(1) <= '0';
        w_A(0) <= '1';
        wait for 1ns;
    end process;
end arch_decodifier2_4_tb;

```

```

assert (w_S(3) = '0') report "Fail 0/1 -> output(3) == 1" severity error;
assert (w_S(2) = '0') report "Fail 0/1 -> output(2) == 1" severity error;
assert (w_S(1) = '1') report "Fail 0/1 -> output(1) != 1" severity error;
assert (w_S(0) = '0') report "Fail 0/1 -> output(0) == 1" severity error;

w_A(1) <= '1';
w_A(0) <= '0';
wait for 1ns;
assert (w_S(3) = '0') report "Fail 1/0 -> output(3) == 1" severity error;
assert (w_S(2) = '1') report "Fail 1/0 -> output(2) != 1" severity error;
assert (w_S(1) = '0') report "Fail 1/0 -> output(1) == 1" severity error;
assert (w_S(0) = '0') report "Fail 1/0 -> output(0) == 1" severity error;

w_A(1) <= '1';
w_A(0) <= '1';
wait for 1ns;
assert (w_S(3) = '1') report "Fail 1/1 -> output(3) != 1" severity error;
assert (w_S(2) = '0') report "Fail 1/1 -> output(2) == 1" severity error;
assert (w_S(1) = '0') report "Fail 1/1 -> output(1) == 1" severity error;
assert (w_S(0) = '0') report "Fail 1/1 -> output(0) == 1" severity error;

wait;
end process;
end arch_decodifier2_4_tb;

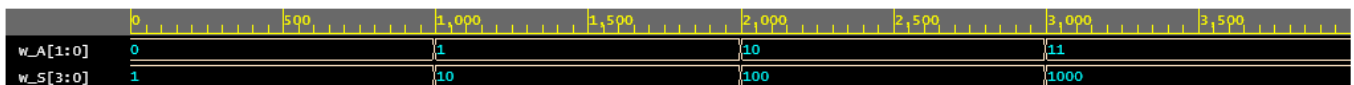
```

Fonte: O autor.

Através do diagrama de forma de onda, é possível ver que o circuito se comporta corretamente:

- A entrada 00 (0) ativa a saída de índice 0 (0001);
- A entrada 01 (1) ativa a saída de índice 1 (0010);
- A entrada 10 (2) gera a saída de índice 2 (0100);
- A entrada 11 (3) gera a saída de índice 3 (1000);

Figura 26 – Diagrama de forma de onda da simulação do decodificador 2-para-4.



Fonte: O autor.

Todos os testes do testbench executam corretamente sem mensagens de erro:

Figura 27 – Imagem do console após a execução do testbench do decodificador 2-para-4.

```
# Aldec, Inc. Riviera-PRO version 2022.04.117.8517 built for Linux64 on May 04, 2022.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2022 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Elaboration time 0.0 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... skipped, nothing to simulate in SLP mode : 0.0 [s]
# SLP: Finished : 0.0 [s]
# ELAB2: You do not have a license to run VHDL performance optimized simulation. Contact Aldec for ordering information - sales@aldec.com.
# ELAB2: Elaboration final pass complete - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 6424 kB (elbread=427 elab2=5854 kernel=142 sdf=0)
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsysstf.so'
# KERNEL: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2022-09-17 21:18:03 EDT] Opening EPWave...
Done
```

Fonte: O autor.